

案例分析考点速记

软考·系统架构设计师

考点速记·打印版

软考系统架构设计师 – 下午 I 案例分析考点速记手册

适用：案例分析(下午 I) 5 道大题 / 满分 75 / 及格 45 / 时长 90 分钟
答题方式：**第 1 题必答** + 后 4 题任选 2 题（共答 3 题，每题 25 分）
编排：基于近 10 年真题考点提炼，并补充近年新增案例热点，覆盖率 $\geq 95\%$

📌 配套：[README.md](#) · [架构设计师考点整理.md](#)

📊 学习优先级（与全套通用图例一致）

标识	含义	案例题应用
P0 ★★★★★	必背	题 1 必答 + 必拿分题型
P1 ★★★★	应知	推荐选答题型核心套路
P2 ★★★	拓展	偶发题型，会比不会强
P3 ★★	了解	罕见题型，可放弃

 案例题必拿分清单 (P0 级)

序号	考点	章节	优先级	适用题
1	6 部分质量属性场景	§2.2	P0	题 1 必答
2	ATAM 4 种点判别	§2.4	P0	题 1 必答
3	效用树标注 H/M/L	§2.5 / §13.10	P0	题 1 必答
4	6 大质量属性战术	§2.3 / §8.1	P0	题 1 必答
5	23 设计模式识别	§5.2	P0	题 4
6	类图关系符号	§5.4	P0	题 4
7	E-R 转关系模式	§4.2 ①	P0	题 3
8	范式判定与分解	§4.2 ③	P0	题 3
9	缓存三大问题	§3.2 ② / §8.5	P0	题 2
10	微服务 vs SOA	§3.2 ④	P0	题 2

选题策略：题 1 必答 → 优先选 **题 4 + 题 3**（套路深、客观性强）→ 实在不会再考虑题 2/5

题型与考频总览（必看）

题号	类型	是否必答	历年高频考点	难度
题 1	软件架构设计与评估	必答	架构风格、质量属性、ATAM、效用树	★★★★★
题 2	Web/分布式架构	选答	微服务、缓存、消息队列、Web 安全	★★★★★
题 3	数据库系统	选答	E-R 转换、规范化、SQL、分布式数据库	★★★★
题 4	UML/设计模式	选答	类图补全、模式应用、代码补全	★★★
题 5	嵌入式/系统集成	选答	实时调度、SOA、ESB、消息中间件	★★★

选题策略：

- 题 1 必答（占 25 分），稳拿 15+ 分
- 后 4 题选 2，**强烈推荐选题 4（设计模式/UML）+ 题 3（数据库）**——客观性强、套路深
- 嵌入式（题 5）通常专业性强，无相关背景慎选

一、答题通用策略（必看）

1.1 时间分配

阶段	时长	任务
审题选题	5 min	5 道题快速浏览，选 2 道熟悉的
题 1	25 min	必答，架构设计
选答题 1	25 min	优先选有把握的
选答题 2	25 min	次选
检查	10 min	通读、补漏

1.2 答题原则

1. **答案在题干中**：80% 答案可从题目描述中提取关键词
2. **使用专业术语**：如"高内聚低耦合""松耦合""权衡点"
3. **分点作答**：用 ①②③ 或 1./2./3. 编号，便于阅卷
4. **先答关键词、再展开**：阅卷按关键词给分
5. **图要规范**：UML 图、E-R 图严格按标准符号绘制
6. **不会的也要写**：写满总分点（每题 5-6 个空，至少猜 3-4 个）

1.3 通用答题套话库

架构选择类：

- "本系统采用 XX 架构风格，因为..."
- "XX 架构的优点是 ①松耦合 ②可扩展 ③易维护"

质量属性类：

- "为提高 XX 性，可采用 XX 战术：①缓存 ②冗余 ③异步"

对比类：

- 列表：维度 / 方案 A / 方案 B / 推荐
 - 结论："综合考虑 XX，建议选择 XX 方案，原因是..."
-

二、题1：软件架构设计与评估（必答，最重要）

2.1 历年真题考点统计

年份	考查主题
2015	软件架构风格 + 决策方法论
2016	质量属性 + 风险点/敏感点
2017	软件架构评估 + 效用树
2018	架构风格选择 + 集成方案
2019	ATAM 评估 + 质量属性场景
2020	架构风格优缺点 + 质量属性战术
2021	风险点/敏感点/权衡点识别
2022	架构评估 + 微服务 + 容器
2023	质量属性 + 架构选择
2024 上	微服务 vs 单体优缺点 + 效用树补全 + 6 部分质量属性场景描述
2024 下	质量属性分类填空 + Ping Echo vs 心跳模式对比（节约资源角度）
2025 上	金融鸿蒙自主可控架构（端-边-云算力分配、TEE 安全防护、运维能力）
2025 下	大模型训练平台 / 数据架构（质量属性映射、架构风格选择）

2.2 必背：6 部分质量属性场景（核心）

每个质量属性场景由 6 个部分构成：

编号	名称	含义	示例
1	刺激源 Source	刺激的发源地	用户、内部系统、外部攻击者
2	刺激 Stimulus	到达系统的事件	请求、故障、攻击
3	制品 Artifact	被刺激的部分	系统、数据库、模块
4	环境 Environment	系统运行状态	正常运行、降级、过载
5	响应 Response	系统采取的动作	处理、记录、拒绝
6	响应度量 Measure	量化指标	响应时间≤2秒、可用性≥99.99%

必考题型：给一段描述，要求填入 6 部分的对应内容或判断属于哪个质量属性

示例题（2019 真题改编）：

"在系统正常运行情况下，系统在 1 秒钟内处理来自外部用户的 1000 个并发请求"

- 刺激源：外部用户
- 刺激：1000 个并发请求
- 制品：系统
- 环境：正常运行
- 响应：处理请求
- 响应度量：1 秒钟内 / 1000 并发
- → 这是**性能**质量属性场景

2.3 6 大质量属性详解

① 可用性 Availability

关注：系统正常服务时间比例 **度量：**MTTF / MTBF / MTTR / 可用性百分比（99.9%、99.99%）

典型战术（必背）：

- **错误检测：**命令/响应、心跳(Heartbeat)、异常检测
- **错误恢复：**表决、主动冗余（热备）、被动冗余（温备/冷备）、备件、回滚、重试、降级
- **错误预防：**从服务中删除（清除）、事务、进程监视器

关键词识别：故障、宕机、恢复、容错、冗余、备份、心跳

② 性能 Performance

关注：响应时间、吞吐量 **度量：**延迟、TPS、QPS、并发数

战术：

- **资源需求：**减少计算开销、降低事件率、限制执行时间、限制队列大小
- **资源管理：**引入并发、维持多个副本（数据/计算）、增加可用资源
- **资源仲裁：**调度策略（FIFO、固定优先级、动态优先级、SJF）

关键词：响应时间、吞吐量、延迟、并发、性能瓶颈、缓存、负载均衡、CDN、异步

③ 安全性 Security

关注：抵抗攻击、保护数据 **度量：**检测/抵抗攻击的能力、恢复时间

战术：

- **抵抗攻击：**用户认证、用户授权、维护数据机密性/完整性、限制暴露、限制访问

- **检测攻击**：入侵检测系统 IDS
- **从攻击中恢复**：恢复=可用性战术；识别攻击者=审计追踪

关键词：认证、授权、加密、防火墙、入侵检测、审计、CA 证书

④ 可修改性 Modifiability

关注：变更的成本和难度 **度量**：变更时间、变更成本、变更影响范围

战术：

- **局部化变更**：维持高内聚、抽象通用服务、预期期望变更
- **防止连锁反应**：信息隐藏、维持现有接口、限制通信路径、使用中介
- **推迟绑定时间**：运行时注册、配置文件、多态、组件替换、参数化

关键词：扩展、修改、新增模块、配置化、解耦、热插拔

⑤ 可测试性 Testability

战术：内置监视器、记录/回放、接口实现分离、依赖注入

关键词：单元测试、自动化、Mock、覆盖率

⑥ 易用性 Usability

战术：分离用户接口（MVC）、支持用户主动（取消、撤销、聚集、再使用）

关键词：用户体验、操作便捷、撤销、引导、培训

2.4 必背：ATAM 评估（架构权衡分析方法）

ATAM 9 步骤（每年都可能考）：

阶段一：演示

1. 介绍 ATAM 方法
2. 介绍商业目标
3. 介绍待评估架构

阶段二：调查与分析

4. 识别架构方法（架构风格、模式）
5. 生成质量属性效用树
6. 分析架构方法（针对效用树）

阶段三：测试

7. 集体讨论场景并定优先级
8. 再次分析架构方法

阶段四：报告

9. 展示评估结果

3 大核心输出（必考）：

概念	定义	识别要点
敏感点 Sensitivity Point	影响 单个 质量属性的架构决策	"为提高 X，采用 Y"
权衡点 Tradeoff Point	影响 多个 质量属性的架构决策	"同时影响 X 和 Z"，常是矛盾权衡
风险点 Risk	可能引起质量目标无法达成的决策	"可能导致..."、"存在风险"
非风险点 Non-Risk	经过分析的好的架构决策	"明确合理"、"已验证"

经典识别题套路（必考！）：

题目给若干描述，让你填到敏感点/权衡点/风险点/非风险点

判别口诀：

- 描述涉及"提高/影响 1 个属性" → **敏感点**
- 描述涉及"同时影响多个属性，需权衡" → **权衡点**
- 描述含"可能"、"风险"、"难以确定" → **风险点**
- 描述明确"经过验证"、"合理"、"成熟" → **非风险点**

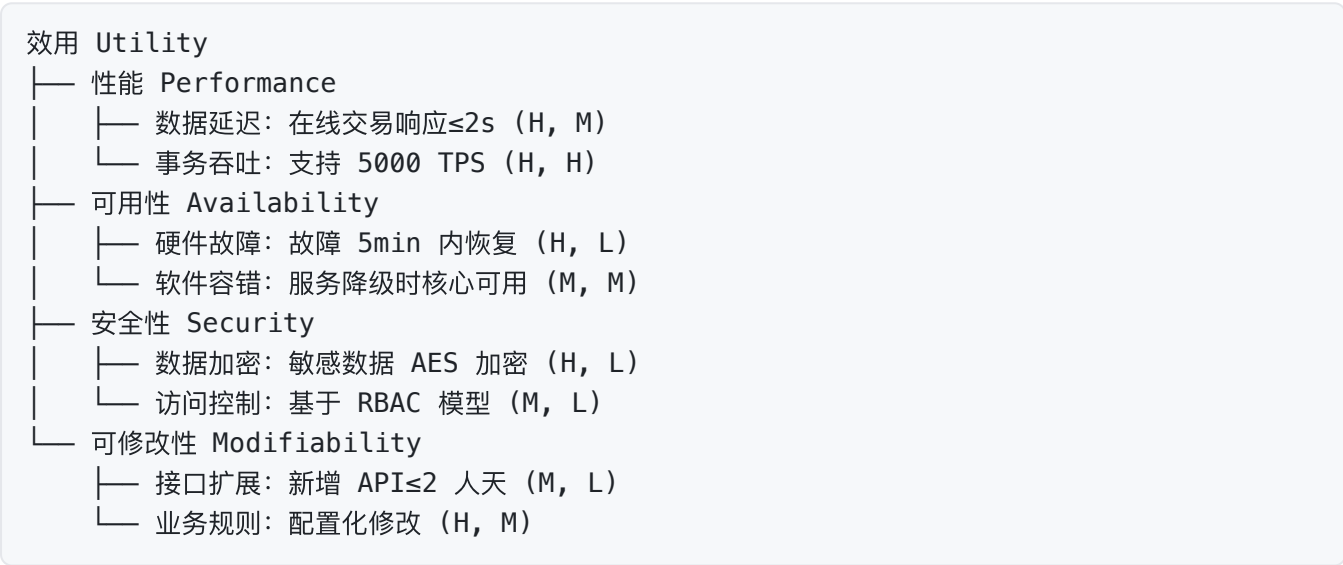
典型例题：

- ① "为提高响应速度，使用 Redis 集群作为缓存层" → **敏感点**（仅影响性能） ② "采用主从同步复制，强一致性同时降低了写入性能" → **权衡点**（同时影响一致性和性能） ③ "暂时无法估计第三方接口的响应时间稳定性" → **风险点** ④ "采用 Nginx 集群进行负载均衡，已在多个项目验证" → **非风险点**

2.5 效用树 (Utility Tree)

结构：根节点(效用) → 质量属性 → 属性细化 → 场景叶子节点 每个叶子标注 (**重要性, 实现难度**): H/M/L

示例：



填空题套路：题目给场景描述，让你画或补全效用树（标 H/M/L）

2.6 SAAM 方法 (架构分析方法)

维度	SAAM	ATAM
提出时间	早(1990s)	晚
主要关注	可修改性、功能性	多个质量属性权衡
评估方式	场景驱动	场景+效用树
适用阶段	架构早期	架构评估全过程

SAAM 步骤：

1. 形成场景（直接场景 / 间接场景）
2. 描述架构
3. 场景分类与确定优先级

4. 单个场景评估
5. 场景交互评估
6. 形成总体评价

直接场景：架构能直接支持 **间接场景：**架构需要修改才能支持（用于评估可修改性）

2.7 CBAM（成本效益分析方法）

- 在 ATAM 之后做经济性评估
- 关注**ROI（投资回报率）**
- 流程：整理场景 → 确定优先级 → 估算收益 → 估算成本 → 计算 ROI → 决策

2.8 题 1 答题模板

典型 5 小问结构：

(1) 简述 XX 架构风格特征/优缺点（5分）

答案模板：从特征、优点、缺点、适用场景 4 方面阐述

(2) 选择/比较两种架构（5分）

答案模板：表格对比 + 结论 + 理由

(3) 识别敏感点/权衡点/风险点/非风险点（10分）

答案模板：题干 5-6 个描述对号入座，按口诀判别

(4) 补充效用树/质量属性场景（5分）

答案模板：按 6 部分场景或效用树结构补全

(5) 实施建议（5分）

答案模板：分点列出技术选型、改造方案、验证方法

三、题2：Web/分布式架构（高频选答）

3.1 历年考点

年份	主题
2017	RESTful + 集群 + 缓存
2018	负载均衡 + Web 安全
2019	微服务 + 容器化
2020	中台架构 + 消息队列
2021	微服务治理 + 服务网格
2022	云原生 + Serverless
2023	分布式缓存 + 一致性
2024 下	数据库缓存策略（Cache Aside 读写流程、双线程不一致问题、三种解决方案）
2024 下	Web 应用（Elasticsearch 四种分词器、RESTful 设计）
2025 上	AI 集群高性能网络架构（RDMA 传输、APSCC 拥塞控制、弹性伸缩、可靠性）

3.2 必备知识点

① 集群与负载均衡

集群类型：

- 高可用集群（HA）：主备/双活，故障切换
- 负载均衡集群（LB）：分摊请求
- 高性能集群（HPC）：并行计算

负载均衡算法：

算法	特点	适用
轮询 RR	顺序分配	节点性能相近
加权轮询 WRR	按权重	节点性能不同
随机	概率分配	简单负载
最小连接数	选连接数少的	长连接业务
最小响应时间	选最快的	性能敏感
一致性哈希	节点变化影响小	缓存集群
IP Hash	同 IP 落同节点	会话保持

LVS 三种模式：

- NAT：通过修改请求目的地址
- DR（直接路由）：性能最好（最常用）
- TUN（IP 隧道）：跨网段

② 缓存

多级缓存：浏览器 → CDN → Nginx → 应用层 → 分布式缓存(Redis) → DB

缓存模式：

- **Cache Aside**（旁路缓存，最常用）：
 - 读：先缓存，未命中读 DB 并写缓存
 - 写：先更新 DB，再删除缓存
- **Read/Write Through**：缓存与 DB 同步
- **Write Behind**：异步写回 DB（高性能但有数据丢失风险）

三大缓存问题（必考）：

问题	原因	解决
缓存击穿	单个热点 key 过期，大量请求打到 DB	互斥锁、永不过期、热点预热
缓存穿透	查询不存在的数据，缓存层无效	布隆过滤器、空值缓存
缓存雪崩	大量 key 同时失效	错峰过期（随机TTL）、多级缓存

Redis 数据类型：String、List、Hash、Set、Sorted Set、Bitmap、HyperLogLog、Stream

Redis 持久化：

- RDB（快照）：定时快照，恢复快但可能丢数据
- AOF（日志）：每次操作追加，丢失少但文件大
- 混合持久化（Redis 4.0+）

Redis 集群方案：

- 主从复制 + 哨兵
- Cluster 集群（哈希槽 16384）
- Codis（豌豆荚开源）

③ 消息队列

作用（必背三要素）：

1. **解耦**：生产者消费者独立
2. **异步**：提高响应速度
3. **削峰**：应对突发流量

主流 MQ 对比：

MQ	吞吐	延迟	特性	适用
Kafka	极高(百万级)	ms-s	日志型、分区有序	大数据、日志
RocketMQ	高(十万级)	ms	事务消息、阿里	金融、电商
RabbitMQ	中(万级)	μs	AMQP、灵活路由	企业、可靠传输
ActiveMQ	较低	ms	JMS、老牌	传统企业

消息可靠性：

- 不丢失：持久化 + ACK + 重试
- 不重复：幂等性、去重表、Token 机制
- 顺序消息：单分区、Hash 分区

④ 微服务架构

核心特征：

1. 服务独立部署
2. 去中心化治理
3. 围绕业务能力组织
4. 智能端点和哑管道
5. 失败设计（Design for Failure）
6. 演进式设计

SOA vs 微服务对比（必考辨析）：

维度	SOA	微服务
通信	ESB（智能管道）	HTTP/RPC（哑管道）
数据	共享数据库	每服务独立数据库
粒度	粗	细
部署	集中部署	独立部署
治理	集中	去中心化
复用度	高	低（接受冗余）
难度	中	高（运维复杂）

微服务治理组件：

类别	主流方案
注册发现	Eureka、Nacos、Consul、ZooKeeper
配置中心	Apollo、Nacos、Spring Cloud Config
网关	Spring Cloud Gateway、Zuul、Kong、APISIX
熔断限流	Hystrix、Sentinel、Resilience4j
调用追踪	Zipkin、SkyWalking、Jaeger
服务网格	Istio、Linkerd、Consul Connect

服务拆分原则：

- 单一职责
- 业务边界（DDD 限界上下文）
- 高内聚低耦合
- 团队规模匹配（two-pizza team）

⑤ Web 安全

OWASP Top 10（2021 标准版，高频）：

编号	项目	典型防御
A01	失效的访问控制 Broken Access Control	RBAC、最小权限、强制服务端校验
A02	加密机制失效 Cryptographic Failures	TLS 1.3、强密钥管理、敏感数据不明文存储
A03	注入 Injection (SQL/XSS/命令注入)	参数化查询、输入校验、输出转义
A04	不安全设计 Insecure Design	威胁建模、安全设计模式、安全开发生命周期
A05	安全配置错误 Security Misconfiguration	硬化基线、最小化默认配置、定期审计
A06	易受攻击和过时的组件 Vulnerable and Outdated Components	SCA 扫描、依赖更新、白名单管理
A07	身份识别和身份验证失败 Identification and Authentication Failures	MFA、强密码策略、防爆破限流
A08	软件和数据完整性失败 Software and Data Integrity Failures	数字签名验证、CI/CD 完整性校验、防反序列化漏洞
A09	安全日志和监控失败 Security Logging and Monitoring Failures	集中日志、SIEM、关键操作审计
A10	服务器端请求伪造 SSRF Server-Side Request Forgery	URL 白名单、网段隔离、禁用危险协议

⚠ 常见易错点：CSRF（跨站请求伪造）已**不在 OWASP 2021 Top 10 单列**，被合并到 A01（失效的访问控制）类目下。考试中如题干提到“防 CSRF”，对应 A01 类防御（Token 校验、SameSite Cookie、Referer 校验）。

关键变化（OWASP 2017 → 2021）：

- 新增：A04 不安全设计、A08 软件和数据完整性失败、A10 SSRF
- 合并/重组：CSRF / XXE 不再单列；XSS 合入 A03 注入

⑥ 容器与云原生

Docker：

- 镜像（image）：只读模板
- 容器（container）：镜像运行实例
- Dockerfile：构建脚本

- Docker Compose：单机多容器编排

Kubernetes 核心概念：

概念	作用
Pod	最小调度单位（一个或多个容器）
Deployment	无状态服务管理
StatefulSet	有状态服务（如数据库）
DaemonSet	每节点跑一个
Service	服务发现与负载均衡
Ingress	七层路由
ConfigMap	配置
Secret	敏感信息
PV/PVC	持久化存储
Namespace	资源隔离

云原生 12 要素（CNCF）：基准代码、依赖、配置、后端服务、构建-发布-运行、进程、端口绑定、并发、易处理、开发/生产环境一致、日志、管理进程

3.3 题 2 答题模板

典型问法 1：选型对比题（如选 Redis 还是 Memcached）

- 表格对比 → 结论 → 理由

典型问法 2：架构改造题（如单体改微服务）

- 现状问题 → 改造方案 → 优势 → 风险 → 应对

典型问法 3：性能/安全/可用性方案

- 问题分析 → 战术列举（缓存/CDN/异步/...） → 选择理由
-

四、题3：数据库系统（最稳定，强烈推荐）

4.1 历年考点

年份	主题
2015–2018	E–R 图补全 + 关系模式转换 + SQL
2019	反规范化 + 性能优化
2020	分布式数据库 + CAP
2021	数据仓库 + 数据湖
2022	分库分表 + 读写分离
2023	NewSQL + HTAP
2024 上	分布式数据管理（数据分片策略、跨库事务）
2024 上	大数据系统架构（Lambda / Kappa、数据湖 vs 数仓）
2025 上	大模型训练数据平台（多源数据集成、存算分离、训练数据治理）

4.2 必备知识点

① E–R 图补全（高频）

实体（矩形）+ 属性（椭圆）+ 联系（菱形）

联系基数：1:1、1:N、M:N

E–R 转关系模式规则：

联系类型	转换规则
1:1	任一方加另一方主键作为外键，或合并
1:N	N 端加 1 端主键作为外键
M:N	必须独立成关系，主键 = 双方主键组合
多元联系	独立成关系，主键 = 各方主键组合
弱实体	独立成关系，主键 = 自身部分键 + 强实体主键

填空套路：题目给场景，要求：

- 补充缺失的实体或联系
- 给出关系模式（含主键、外键）

- 标识候选键

② SQL 题型

经典题型：

1. 创建表（含 PK、FK、约束）
2. 复杂查询（嵌套、关联、聚合）
3. 视图创建
4. 触发器/存储过程

SQL 关键字必背：

```
-- 创建表
CREATE TABLE 表名 (
    列名 类型 PRIMARY KEY,
    列名 类型 NOT NULL,
    列名 类型 UNIQUE,
    列名 类型 DEFAULT 值,
    列名 类型 CHECK (条件),
    FOREIGN KEY (列名) REFERENCES 主表(主键)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

-- 复杂查询模板
SELECT 列, 聚合函数(列)
FROM 表1 [INNER|LEFT|RIGHT] JOIN 表2 ON 条件
WHERE 过滤条件
GROUP BY 列
HAVING 聚合条件
ORDER BY 列 [ASC|DESC]
LIMIT N;

-- 视图
CREATE VIEW 视图名 AS
SELECT ... FROM ... WHERE ...;
```

子查询：

- 标量子查询（返回单值）
- 列子查询（IN / NOT IN / ANY / ALL）
- 行子查询（= / <>）
- 表子查询（FROM 后）
- 关联子查询（EXISTS / NOT EXISTS）

③ 范式与函数依赖

判定流程：

1. 找出所有函数依赖
2. 求候选键（三步法）
3. 判断范式

范式判定：

- **1NF**：每个属性不可再分
- **2NF**：1NF + 非主属性完全依赖于码（消除部分依赖）
- **3NF**：2NF + 非主属性不传递依赖于码（消除传递依赖）
- **BCNF**：3NF + 主属性也无部分/传递依赖（左侧均含码）

模式分解：

- **无损连接**：分解后通过自然连接能恢复（**必须**保证）
- **保持函数依赖**：分解后所有原依赖仍成立（**最好**保证）

必考二元分解判定：R 分解为 R1, R2 是无损连接 $\iff$

- $R1 \cap R2 \rightarrow R1 - R2$, 或
- $R1 \cap R2 \rightarrow R2 - R1$

④ 反规范化

何时反规范化：

- 查询性能瓶颈
- 报表/统计场景
- 读多写少

常用手段：

1. 增加冗余列（避免连接）
2. 增加派生列（预计算结果）
3. 重新组表（合并小表）
4. 分表（水平/垂直）

反规范化代价：数据冗余、维护一致性成本高 $\rightarrow$ 通过触发器/应用层维护

⑤ 分布式数据库

数据分片：

- **水平分片**（按行）：按某列范围/哈希拆分

- **垂直分片**（按列）：按业务域拆分

分库分表策略：

- 分片键选择：访问频繁、分布均匀
- 分片算法：范围、哈希、一致性哈希

分布式事务：

- 2PC：准备 + 提交，强一致但阻塞
- 3PC：CanCommit + PreCommit + DoCommit
- TCC（Try-Confirm-Cancel）：业务补偿
- Saga：长事务，本地事务串联+补偿
- 最终一致性：消息表、事件溯源

分布式数据库分布性透明：

- 分片透明：用户不感知分片
- 复制透明：用户不感知副本
- 位置透明：用户不感知物理位置
- 逻辑透明：用户不感知数据模型差异

⑥ 数据仓库与大数据

OLTP vs OLAP：

特性	OLTP	OLAP
用户	业务人员	决策者
操作	增删改查	复杂分析
数据量	GB	TB-PB
事务	短小、并发高	长、并发低
设计	规范化	反规范化（星型/雪花）

ETL：抽取 → 转换 → 加载

数据仓库建模：

- **星型模型**：1 张事实表 + N 张维度表（最常用）
- **雪花模型**：维度表进一步规范化（节省空间但查询慢）
- **星座模型**：多张事实表共享维度

数据湖 vs 数据仓库：

维度	数据仓库	数据湖
数据	结构化	原始（结构化+半结构化+非结构化）
Schema	Schema-on-Write	Schema-on-Read
用户	业务分析师	数据科学家
成本	高	低

Lambda 架构：批处理层 + 速度层 + 服务层（双路径） **Kappa 架构：**仅流处理（简化）

4.3 题 3 答题模板

经典 5 小问结构：

- 1. **补充 E-R 图实体/联系：**根据题目描述识别遗漏
 - 2. **关系模式补全（含主键外键）：**按转换规则
 - 3. **指出候选键：**用三步法
 - 4. **SQL 编写：**仔细按需求拼接
 - 5. **优化建议：**索引、反规范化、分库分表
-

五、题4：UML与设计模式（强烈推荐选答）

5.1 历年考点

年份	主题
2015	观察者模式 + 类图补全
2016	装饰器模式 + 代码补全
2017	策略模式 + UML 关系
2018	工厂方法 + 抽象工厂
2019	命令模式 + 时序图
2020	适配器模式 + 状态图
2021	组合 + 迭代器
2022	桥接模式
2023	责任链 + 命令
2024 上	面向对象分析与设计（OOA/OOD 综合 + 类图设计 + UML 关系判断）
2024 下	风险分析 + UML（糖尿病胰岛素场景的危险驱动分析 4 过程 + 状态图/类图补全）

题型套路（雷打不动）：

- (1) 给一段场景描述，要求识别**最适合的设计模式**及其理由
- (2) 补全类图（缺失的类、关系、属性、方法）
- (3) **代码填空**（Java/C++ 代码片段，填关键语句）
- (4) 设计模式优缺点对比

5.2 23 种设计模式速查（必背）

创建型（5 个）

1. 单例 Singleton

- 应用：连接池、配置管理、日志
- 实现要点：私有构造、静态实例、获取实例方法

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

2. 工厂方法 Factory Method

- 应用：JDBC、日志框架
- 关键词："让子类决定实例化哪个类"

```
abstract class Creator { abstract Product factoryMethod(); }
class ConcreteCreatorA extends Creator { Product factoryMethod(){return new ProductA();} }
```

3. 抽象工厂 Abstract Factory

- 应用：跨平台 UI、主题切换
- 关键词："产品族"、"多个相关产品"

4. 建造者 Builder

- 应用：StringBuilder、构造复杂对象
- 关键词："分步构建"、"Director + Builder"

5. 原型 Prototype

- 应用：对象克隆、Java 的 Cloneable
- 关键词："克隆"、"复制创建"

结构型 (7 个)

6. 适配器 Adapter

- 应用：旧接口适配新接口、Stream/Reader 适配
- 关键词："接口转换"
- 类适配器（继承）vs 对象适配器（组合，更常用）

7. 桥接 Bridge

- 应用：跨平台、形状-颜色独立变化
- 关键词："抽象与实现分离"、"两个变化维度"

8. 组合 Composite

- 应用：文件系统、组织架构、菜单
- 关键词："树形结构"、"部分-整体"
- 一致对待叶子节点和组合节点

9. 装饰器 Decorator

- 应用：Java IO 流、日志增强
- 关键词："动态添加职责"、"包装"
- 与继承的区别：动态、灵活、不污染原类

10. 外观 Facade

- 应用：库的统一入口、SDK
- 关键词："简化接口"、"门面"

11. 享元 Flyweight

- 应用：字符串常量池、棋子复用、连接池
- 关键词："共享细粒度"、"内部状态/外部状态"

12. 代理 Proxy

- 应用：远程代理（RPC）、虚代理（懒加载）、保护代理（权限）、动态代理（Spring AOP）
- 关键词："代理访问"

行为型（11 个）

13. 责任链 Chain of Responsibility

- 应用：审批流、过滤器链（Filter）、Servlet
- 关键词："链式处理"、"多个对象处理请求"

14. 命令 Command

- 应用：撤销/重做、任务队列、按钮事件
- 关键词："请求封装为对象"、"撤销"
- 角色：Invoker、Command、Receiver

15. 解释器 Interpreter

- 应用：DSL、规则引擎、SQL 解释
- 关键词："文法解释"

16. 迭代器 Iterator

- 应用：集合遍历
- 关键词："遍历不暴露内部"

17. 中介者 Mediator

- 应用：聊天室、MVC 控制器、机场塔台
- 关键词："通过中介通信"、"减少多对多耦合"

18. 备忘录 Memento

- 应用：撤销操作、游戏存档
- 关键词："保存恢复状态"、"快照"
- 角色：Originator、Memento、Caretaker

19. 观察者 Observer

- 应用：发布订阅、事件监听、MVC 中 Model→View
- 关键词："一对多依赖"、"自动通知"
- 推模型 vs 拉模型

20. 状态 State

- 应用：订单状态机、TCP 连接状态、电梯
- 关键词："对象行为随状态改变"
- vs 策略：状态会自动切换、强调状态转换

21. 策略 Strategy

- 应用：促销算法、支付方式、排序算法
- 关键词："算法可替换"、"策略族"
- vs 状态：策略由客户端选择、不会自动切换

22. 模板方法 Template Method

- 应用：钩子方法、抽象类基础流程
- 关键词："算法骨架"、"子类重写步骤"

23. 访问者 Visitor

- 应用：编译器 AST 处理、复杂数据结构操作
- 关键词："双分派"、"不修改类增加新操作"
- 角色：Visitor、Element、ObjectStructure

5.3 易混设计模式辨析（必考）

对比	区别
工厂方法 vs 抽象工厂	前者一个产品等级；后者 产品族 （多等级）
适配器 vs 装饰器	适配器 改接口 ；装饰器 加功能 （接口不变）
适配器 vs 桥接	适配器 事后 ；桥接 事前 设计
代理 vs 装饰器	代理 控制访问 ；装饰器 增强功能
策略 vs 状态	策略 客户端选择 ；状态 自动切换
策略 vs 模板方法	策略 组合 （运行时换）；模板 继承 （编译时）
观察者 vs 中介者	观察者 一对多直接通知 ；中介者 多对多经中介
命令 vs 策略	命令 封装请求 （带历史）；策略 封装算法
组合 vs 装饰器	组合 树形结构 ；装饰器 线性嵌套
外观 vs 中介者	外观 单向 简化；中介者 双向 协调

5.4 UML 类图核心要点

类图三要素：

+-----+

| 类名 |

+-----+

| - 属性：类型 | (- 私有, + 公有, # 保护, ~ 包)

+-----+

| + 方法(): 类型 |

+-----+

关系符号（必考）：

关系	符号	实例
依赖	虚线箭头 →---→	方法参数、局部变量
关联	实线箭头 →→	类成员变量
聚合	空心菱形 ◇—	部门-员工（员工可属其他）
组合	实心菱形 ◆—	人-心脏（同生死）
泛化	空心三角 △—	子类→父类
实现	虚线空心三角 △---	类→接口

类图填空套路：

- 给定类和职责描述
- 要求标注：类间关系、关键属性、关键方法、多重性（1, 0..1, , 1..）

5.5 时序图与状态图

时序图（Sequence Diagram）：

- 强调**时间顺序**
- 元素：参与者（生命线）、消息（同步/异步/返回）、激活条
- 题型：补全消息流向

状态图（State Diagram）：

- 描述对象**状态变化**
- 元素：初始状态●、状态、转换、终止状态◎
- 题型：补全状态、触发事件、动作

5.6 代码填空高频套路

模式代码识别口诀：

模式	关键代码特征
单例	<code>getInstance()</code> , <code>private static instance</code>
工厂方法	<code>abstract createProduct()</code> , 子类返回不同 Product
装饰器	持有被装饰对象引用, 方法中调用并增强
适配器	持有被适配对象, 方法名对接新接口
代理	持有真实对象引用, 调用前后做拦截
观察者	<code>attach/detach</code> 、 <code>notify()</code> 、 <code>update()</code>
策略	Context 持有 Strategy 接口引用
模板方法	抽象类含 <code>final</code> 模板 + 多个抽象步骤
命令	<code>execute()</code> 方法、Invoker 持有 Command
状态	Context 持有 State 引用, State 内 <code>setState</code> 切换
责任链	持有 <code>nextHandler</code> 引用, 递归调用

5.7 题 4 答题模板

典型 5 小问结构：

1. **场景适合的设计模式**：识别 + 名称 + 模式分类（创/结/行）
 2. **设计模式的优点 / 适用场景**：3-4 条要点
 3. **类图补全**：标关系、多重性、属性、方法
 4. **代码填空**：按模式标准代码套路
 5. **若需求变化，模式如何应对**：体现模式的扩展性
-

六、题5：嵌入式/系统集成（专业性强，谨慎选答）

6.1 历年考点

年份	主题
2015–2017	实时系统 + 任务调度
2018	嵌入式数据库
2019	ESB + 服务编排
2020	工作流引擎 + BPM
2021	物联网架构
2022	边缘计算
2023	数字孪生
2024 上	嵌入式技术（实时调度 + 硬实时设计）
2024 下	ROS / ROS2 机器人操作系统（ROS 定义、ROS2 改进方向、四种通信方式）
2025 上	金融鸿蒙自主可控（鸿蒙生态适配、端-边-云算力、TEE 可信执行环境）

6.2 嵌入式系统知识点

实时系统分类：

- **硬实时**：必须在截止期前完成（医疗、航天、工业控制）
- **软实时**：偶尔超时可接受（视频流、游戏）

嵌入式系统层次：

1. 硬件层：处理器（ARM/MIPS/PowerPC）+ 外设
2. 中间层（BSP）：板级支持包，硬件抽象层
3. 系统软件层：嵌入式 OS + 文件系统 + 网络协议栈
4. 应用软件层

嵌入式 OS 特性：

- 可剪裁性：按需配置
- 强实时性
- 高可靠性
- 易移植性
- 低资源消耗

任务调度算法：

算法	类型	公式/特点
RM（速率单调）	静态优先级	周期短优先级高
EDF（最早截止期）	动态优先级	截止期近优先
LLF（最低松弛度）	动态	松弛度=截止时间-当前-剩余执行

RM 可调度判据： CPU 利用率 $\leq n(2^{1/n} - 1)$

- $n=1$: 100%
- $n=2$: 82.8%
- $n=3$: 77.9%
- $n=\infty$: 69.3%

EDF 可调度判据： CPU 利用率 $\leq 100\%$

优先级反转： 低优先级持有锁阻塞高优先级 解决：

- **优先级继承协议：** 低优先级临时继承高优先级
- **优先级天花板协议：** 访问资源时直接提到最高优先级

6.3 系统集成

集成层次：

- 数据集成：ETL、数据复制
- 应用集成：消息、API、服务
- 业务集成：流程编排
- 表示集成：门户

EAI 集成模式：

- 文件传输：批量、松耦合
- 共享数据库：实时、紧耦合
- RPC：实时、紧耦合
- 消息：异步、松耦合（**最推荐**）

ESB（企业服务总线）：

- 核心功能：消息路由、协议转换、消息转换、安全、监控
- 模式：服务总线，集中式
- 主流产品：Mule、WSO2、Apache ServiceMix、阿里 EDAS

SOA 治理：

- 设计时治理：标准、规范
- 运行时治理：监控、SLA
- 服务版本管理
- 服务生命周期管理

6.4 工作流与 BPM

WfMC 工作流参考模型（5 个接口）：

1. 流程定义工具
2. 工作流客户端应用
3. 调用应用
4. 其他工作流引擎
5. 管理与监控工具

BPM 核心概念：

- BPMN（业务流程建模标记语言）
- BPEL（业务流程执行语言）
- 流程编排（Orchestration）vs 流程协同（Choreography）

6.5 物联网架构

三层架构：

1. **感知层**：传感器、RFID、二维码、GPS
2. **网络层**：通信网络（5G、NB-IoT、LoRa、Zigbee）
3. **应用层**：行业应用（智慧城市、工业互联网）

也可分四层：感知层 + 网络层 + 平台层 + 应用层

物联网协议：

- MQTT（轻量、发布订阅）
- CoAP（受限设备）
- LwM2M（设备管理）

边缘计算：

- 在数据源附近处理，降低延迟
- 与云计算协同：边缘做实时处理，云端做分析

6.6 题 5 答题模板

实时系统题型：

- (1) 给出任务参数（周期、执行时间、截止期），判断是否可调度
- (2) 设计调度算法
- (3) 解决优先级反转

集成题型：

- (1) 选择集成方式（数据/应用/业务）
- (2) 选择中间件（消息队列、ESB、API 网关）
- (3) 设计集成方案的优缺点

七、综合答题套路（万能模板）

7.1 "为什么选择 X 架构/方案" 模板

本系统选择 [架构/方案] 的原因：

① 业务匹配：业务特征 [描述] 与该架构的优势 [描述] 匹配

② 质量属性：满足系统对 [性能/可用性/...] 的要求

③ 技术成熟：业界已有成功案例（举例）

④ 团队匹配：团队具备相关技术储备

⑤ 成本可控：[成本分析]

但需注意以下风险：

- 风险 1：[描述] → 应对：[措施]

- 风险 2：[描述] → 应对：[措施]

7.2 "比较 X 和 Y 方案" 模板

对比维度	方案 X	方案 Y
性能	[优劣]	[优劣]
可用性	[优劣]	[优劣]
扩展性	[优劣]	[优劣]
成本	[优劣]	[优劣]
复杂度	[优劣]	[优劣]

综上，建议选择 [方案]，因为 [关键原因]

7.3 "提高 XX 性的方案" 模板

性能优化：

- 前端：CDN、压缩、合并请求
- 网络：HTTP/2、KeepAlive
- 应用：异步、并发、连接池
- 缓存：浏览器→Nginx→Redis→DB
- 数据库：索引、读写分离、分库分表
- 架构：水平扩展、负载均衡

可用性方案：

- 冗余设计（主备、集群）
- 容错（熔断、降级、限流）
- 监控告警（健康检查、心跳）
- 灾备（异地多活）

安全性方案：

- 网络层：防火墙、WAF、DDoS 防护
- 应用层：认证、授权、加密、审计
- 数据层：脱敏、加密存储、备份
- 运维层：补丁、漏洞扫描、应急响应

7.4 "识别风险点/敏感点/权衡点" 模板

针对题目给出的若干描述，逐条判别：

[描述1]：仅影响 [某个属性]，且为关键决策 → 敏感点

[描述2]：同时影响 [属性A] 和 [属性B]，需在二者间权衡 → 权衡点

[描述3]：含"可能"、"未确定"、"难以估计" → 风险点

[描述4]：明确合理且有验证 → 非风险点

八、必背速查表

8.1 6 大质量属性 + 战术速查

可用性 → 检测(心跳/Ping)、恢复(冗余/回滚)、预防(事务/监视)
性能 → 减需求、管资源(并发/副本)、调度(FIFO/优先级)
安全性 → 抵抗(认证/授权/加密)、检测(IDS)、恢复(审计)
可修改 → 局部(高内聚)、防连锁(信息隐藏)、推迟绑定(配置/插件)
可测试 → 内置监视、记录回放、接口分离
易用性 → 分离UI、用户主动(撤销/取消)

8.2 12 种架构风格速查

数据流：批处理、管道-过滤器
调用：主程序-子程序、OO、层次、C/S、B/S
独立：进程通信、事件驱动
虚拟机：解释器、规则
仓库：数据库、黑板、超文本
现代：SOA、微服务、REST、Serverless

8.3 ATAM 关键名词速查

敏感点：影响【1 个】质量属性的决策
权衡点：影响【多个】质量属性的决策
风险点：可能影响目标达成的【未确定】决策
非风险点：经过验证的合理决策

8.4 23 种设计模式速查

创建型(5)：单工抽建原 = 单例 + 工厂方法 + 抽象工厂 + 建造者 + 原型
结构型(7)：适桥组装外享代 = 适配器 + 桥接 + 组合 + 装饰 + 外观 + 享元 + 代理
行为型(11)：策观模命状责迭中备访解
= 策略+观察者+模板方法+命令+状态+责任链+迭代器+中介者+备忘录+访问者+解释器

8.5 缓存三大问题速查

击穿：单 key 失效 → 互斥锁、永不过期
穿透：查不存在 key → 布隆过滤器、空值缓存
雪崩：大量同时失效 → 错峰过期、多级缓存

九、近 5 年真题分析

 **题型说明：**案例分析一直是 5 道大题，第 1 题必答 + 后 4 题任选 2 题（共答 3 题，每题 25 分）。下方列表展示全部 5 道题（回忆版），考生在场上按"必答 + 选答"策略作答即可。

2025 年案例分析（回忆版）

2025 年 11 月（下半年）回忆版

题号	主题	关键考点
题 1	软件架构评估	质量属性场景、敏感点/权衡点/风险点判别
题 2	金融鸿蒙自主可控架构	鸿蒙生态适配、端-边-云算力、TEE 安全防护、运维能力
题 3	AI 集群高性能网络	RDMA、APSCC 拥塞控制、弹性伸缩、网络可靠性
题 4	设计模式 / 数据库	（待考生进一步回忆核实）
题 5	嵌入式 / 大模型应用	（待考生进一步回忆核实）

2025 年 5 月（上半年）回忆版

题号	主题	关键考点
题 1	软件架构评估	在线大模型训练平台，质量属性映射、架构风格选择
题 2	AI 集群高性能网络架构	RDMA 传输、APSCC、弹性伸缩
题 3	数据架构 / 数据库	（待考生回忆核实）
题 4	设计模式	（待考生回忆核实）
题 5	金融鸿蒙自主可控	鸿蒙生态、端-边-云算力分配、TEE 安全

 上述 2025 年表格依据公开回忆版整理，部分题号顺序可能有出入；以官方真题为准。

2024 年案例分析

2024 年 11 月（下半年）

题号	主题	关键考点
题 1	质量属性	分类填空（功能/性能/安全）、Ping Echo vs 心跳节约资源对比
题 2	数据库缓存策略	Cache-Aside 读写流程图、双线程不一致问题 + 3 种解决方案
题 3	Web 应用	Elasticsearch 四种分词器、RESTful 设计
题 4	风险分析	糖尿病胰岛素场景的危险驱动分析 4 过程
题 5	ROS 嵌入式	ROS 定义、ROS2 改进、4 种通信方式

2024 年 5 月（上半年）

题号	主题	关键考点
题 1	架构设计与评估	微服务 vs 单体优缺点、效用树补全、6 部分质量场景
题 2	分布式数据管理	数据分片、跨库事务
题 3	嵌入式技术	硬实时、任务调度
题 4	面向对象分析与设计	OOA/OOD、类图、UML 关系
题 5	大数据系统架构	Lambda 架构、数据湖

2023 年案例分析（参考）

题号	主题	关键考点
题 1	架构评估	质量属性场景识别、敏感点权衡点风险点
题 2	微服务 + 缓存	缓存模式、击穿穿透雪崩
题 3	数据库设计	E-R 转换、SQL、规范化
题 4	设计模式	责任链 + 命令
题 5	物联网架构	三层架构、协议选型

2022 年案例分析

题号	主题	关键考点
题 1	架构评估	ATAM、效用树
题 2	云原生 + 容器	K8s、微服务治理
题 3	分库分表	水平/垂直分片、分布式事务
题 4	设计模式	桥接
题 5	边缘计算	边缘 vs 云端

2021 年案例分析

题号	主题	关键考点
题 1	架构评估	风险点/敏感点
题 2	微服务治理	服务网格、Sidecar
题 3	数据仓库	星型 vs 雪花、ETL
题 4	设计模式	组合 + 迭代器
题 5	工作流	BPM、BPMN

十、近年新增案例热点补充

新热点通常不会脱离架构师主线，本质仍然考：**质量属性、架构取舍、技术选型、风险与应对**。

10.1 Serverless 架构案例

常见题干特征：

- 业务流量波动大，存在明显峰谷
- 希望降低运维成本、按量付费
- 由 HTTP 请求、消息、文件上传、定时任务触发

答题关键词：

- 事件驱动、函数计算、API 网关、BaaS、自动弹性、按量计费
- 状态外置：数据库、缓存、对象存储

- 可观测性：日志、指标、链路追踪

优点：

1. 自动扩缩容，适合突发流量
2. 无需管理服务器，降低运维成本
3. 按实际调用付费，资源利用率高
4. 便于快速开发轻量任务

风险与应对：

1. 冷启动影响响应时间 → 预热、保留实例、拆分轻量函数
2. 厂商锁定 → 采用标准事件模型、隔离云厂商 SDK
3. 函数无状态 → 状态外置到 Redis/数据库/对象存储
4. 调试困难 → 完善日志、链路追踪和本地模拟环境

10.2 云原生数据库 / 存算分离案例

常见题干特征：

- 数据量增长快，读写压力不均衡
- 希望计算资源和存储资源独立扩展
- 要求自动备份、容灾、高可用、低运维成本

答题关键词：

- 存算分离、弹性扩缩容、多副本、自动故障转移、备份恢复
- HTAP、NewSQL、分布式事务、全局一致性

方案模板：

1. 存储层采用分布式共享存储，多副本保证可靠性
2. 计算层无状态化，按业务压力横向扩展
3. 读写分离和只读副本承担读压力
4. 自动备份、跨可用区部署，满足 RPO/RTO
5. 通过监控慢 SQL、连接数、锁等待优化性能

风险与应对：

- 跨节点事务复杂 → 控制事务边界，必要时采用最终一致
- 网络延迟增加 → 就近部署、缓存热点数据
- 成本不可控 → 设置资源配额、自动伸缩阈值和成本告警

10.3 信创 / 国产化适配案例

常见题干特征：

- 政务、金融、能源等关键行业
- 要求国产 CPU、操作系统、数据库、中间件替换
- 关注自主可控、安全合规、平滑迁移

答题框架：

1. **兼容性评估**：梳理硬件、OS、数据库、中间件、第三方组件依赖
2. **分层适配**：基础设施、数据层、应用层、接口层逐层替换
3. **数据迁移**：结构迁移、数据校验、双写或灰度切换
4. **性能验证**：基准测试、压力测试、容量评估
5. **安全合规**：国密改造、等保要求、日志审计
6. **灰度上线**：先非核心业务，再核心业务，保留回退方案

常见风险：

- SQL 方言不兼容
- 驱动和中间件 API 差异
- 性能下降
- 第三方组件不支持国产环境

10.4 端-边-云协同案例

常见题干特征：

- 物联网、工业互联网、视频分析、车联网
- 要求低延迟、断网可用、减少带宽占用
- 终端数量多，数据实时性要求高

职责划分：

层次	职责
端	数据采集、设备控制、轻量推理
边	实时处理、协议转换、缓存、局部决策
云	全局调度、模型训练、海量存储、统一运维

答题要点：

- 边缘节点就近处理，降低时延

- 云端统一管理模型、策略和设备
- 边云断连时本地自治，恢复后数据同步
- 采用 MQTT/CoAP 等轻量协议接入设备

10.5 AI 集群 / 高性能网络案例（了解型）

常见题干特征：

- 大模型训练、AI 推理平台、GPU/NPU 集群
- 关注训练效率、网络拥塞、节点故障恢复

关键词：

- GPU/NPU、RDMA、RoCE、AllReduce、高速互联、弹性调度
- 容器化调度、资源隔离、监控告警、故障迁移

答题方向：

1. 计算层：GPU/NPU 池化，按任务调度资源
2. 网络层：RDMA/RoCE 降低通信延迟
3. 存储层：高吞吐并行文件系统或对象存储
4. 调度层：Kubernetes + AI 调度插件
5. 可靠性：节点健康检测、任务重试、检查点恢复

10.6 性能测试与性能优化案例

常见题干特征：

- 要求设计性能测试方案
- 关注响应时间、吞吐量、并发数、资源利用率

测试类型：

- 负载测试：验证预期负载下能否满足指标
- 压力测试：找系统极限和瓶颈
- 容量测试：评估最大用户数/数据量
- 稳定性测试：长时间运行观察资源泄漏

指标：

- 平均响应时间、P95/P99 响应时间
- TPS/QPS、并发用户数
- CPU、内存、磁盘 I/O、网络 I/O
- 错误率、超时率

优化套路： 前端 CDN/压缩 → 网关限流 → 应用异步/线程池 → 缓存 → 数据库索引/读写分离 → 水平扩展。

10.7 通信系统架构案例（补强）

常见题干特征：

- 出现 5G/工业网关/边缘站点/专网改造等背景
- 要求兼顾低时延、高可靠、可扩展与安全隔离
- 关注协议适配、链路冗余、QoS、故障切换

答题关键词：

- 分层设计（接入层/传输层/控制层/应用层）
- 协议适配（MQTT/CoAP/HTTP/TCP/UDP）
- QoS（时延、抖动、丢包率、带宽保障）
- 高可用（双链路、主备切换、链路探活）
- 安全（TLS、设备认证、访问控制、审计）

标准答题骨架：

1. **架构分层：**说明各层职责，避免耦合到单一协议或单一厂商设备。
2. **接入与协议：**端侧按场景选择轻量协议，网关统一协议转换与数据标准化。
3. **传输与可靠性：**关键链路双活/主备，心跳探测+自动切换，消息重传与去重。
4. **性能与 QoS：**核心业务流量优先级更高，控制类报文优先保障时延。
5. **安全与运维：**设备身份认证、链路加密、日志审计、统一监警告警。

10.8 层次式/信息系统架构案例（补强）

常见题干特征：

- 需要从"业务-应用-数据-技术"视角做整体设计
- 要求说明为什么采用分层，以及层间交互规则
- 经常考察扩展性、可维护性、性能与一致性的权衡

信息系统四层速记：

层次	核心职责	常见输出
业务架构	业务能力、流程、组织职责	业务能力图、流程图
应用架构	系统边界、服务协作、接口	应用清单、接口矩阵
数据架构	主数据、数据流、共享与治理	主题域、数据模型
技术架构	部署、网络、中间件、安全与运维	部署图、技术栈

层次式架构答题要点：

- 只允许相邻层调用，禁止跨层直连数据库
- 将横切关注点（鉴权、日志、缓存、限流）沉淀为公共能力
- 高变更业务逻辑与稳定技术能力解耦，提升可维护性
- 对性能敏感链路可采用"受控旁路"，但必须说明边界和治理措施

十一、考前 7 天案例分析冲刺

D-7 至 D-5：题 1 架构评估

- [] 6 大质量属性 + 6 部分场景
- [] ATAM 9 步骤 + 4 种点（敏感/权衡/风险/非风险）
- [] 12 种架构风格优缺点
- [] 效用树标注 H/M/L

D-4 至 D-3：题 3 数据库 + 题 4 模式

- [] E-R 转关系模式 6 种规则
- [] 范式判定 + 模式分解
- [] SQL 复杂查询模板
- [] 23 种设计模式识别词
- [] 5 个易混模式辨析

D-2：题 2 Web 架构 + 题 5 嵌入式

- [] 缓存三大问题 + 解决
- [] 微服务 vs SOA
- [] 消息队列三大作用
- [] RM/EDF 调度

D-1: 套话与模板

- [] 通用答题套话背 3 遍
- [] 万能模板（选型/对比/提高）默写

考试当天

- 5 道题 90 分钟，平均 18 分钟/题
- 优先答题 1 → 题 4 → 题 3 → 题 2/5
- 不要追求每题满分，追求**每题 15+ 分**
- 字迹工整、分点作答、关键词靠前

十二、2024-2026 题风映射速查（补充）

题干关键词	优先调用知识点	答题落点
"低时延"、"链路抖动"、"多协议接入"	通信系统架构 + QoS	分层、协议适配、链路冗余、指标量化
"业务架构/应用架构/数据架构/技术架构"	信息系统四层架构	层次职责、层间依赖、治理机制
"架构演进"、"历史包袱"、"持续治理"	架构演化与治理	ADR、评审机制、技术债、度量闭环
"高并发 + 稳定性"	性能优化 + 高可用战术	限流/熔断/降级/异步削峰
"国产化迁移"、"兼容风险"	信创适配架构	依赖梳理、分阶段迁移、回退预案

作答建议：

1. 先写结论（选型/方案），再写依据（约束+质量属性）。
2. 每个措施尽量给出 1 个可验证指标（如 P99、RTO、错误率）。
3. 避免只写工具名，要写"机制 + 触发条件 + 预期效果"。

十三、查漏补缺案例补强专题

针对历年真题中**易丢分、易遗漏、易答偏**的高价值案例考点补强。每个专题给出**关键步骤 + 标准答案骨架 + 易错点**。

13.1 数据库设计完整 6 步骤（题 3 必备）

案例题 3 常出"完整数据库设计案例"，要求按规范步骤作答。

步骤	任务	输出物	答题关键词
① 需求分析	收集业务流程与数据需求	需求规格、DFD、数据字典	业务调研、用户访谈
② 概念结构设计	抽象业务实体与关系，与 DBMS 无关	E-R 图	实体、属性、联系、聚合
③ 逻辑结构设计	E-R 转关系模式，按范式优化	关系模式（含主键/外键）	1NF→3NF/BCNF、规范化
④ 物理结构设计	选择存储结构与访问路径	索引、分区、表空间	B+树索引、聚簇
⑤ 数据库实施	建库建表、初始数据加载	DDL、初始数据	建表、约束、应用接入
⑥ 运行与维护	监控、调优、备份	监控报告、备份策略	慢查询、备份恢复、安全

典型考法：

- 给出业务描述 → 补 E-R 图缺失实体/联系
- 给出 E-R → 写出关系模式（标主键、外键、候选键）
- 给出关系模式 → 判断范式、给出分解
- 给出场景 → 编写 SQL（含联接、聚合、子查询）

13.2 优先级反转完整例题（题 5 嵌入式必备）

题干样式：

任务 H（优先级 10）、M（优先级 5）、L（优先级 1）共享互斥资源 R：

- t=0: L 启动，锁定 R，开始执行
- t=2: H 就绪，抢占 L
- t=2.5: H 请求 R，被阻塞，L 继续持锁运行
- t=3: M 就绪，抢占 L 执行（与 R 无关） 分析存在什么问题，提出解决方案。

标准答案骨架：

(1) 问题诊断：

发生**优先级反转**：高优先级 H 因等待 L 释放 R 而阻塞，但中优先级 M 抢占了 L 的执行，导致 H 间接被 M 阻塞，违反优先级调度原则，可能引起 H 错过截止期。

(2) 解决方案对比：

协议	机制	优缺点
优先级继承协议 PIP	L 锁 R 期间，若被 H 阻塞，则 L 临时继承 H 的优先级 → M 无法抢占 L → L 尽快释放 R	实现简单，但可能链式继承
优先级天花板协议 PCP	L 一获得 R 就立即提升到该资源的 最高使用者优先级 （天花板）	防止死锁与链式继承，但要预知最高优先级

(3) 工程建议：

- 缩短临界区
- 避免嵌套加锁
- 使用支持 PIP 的 RTOS（VxWorks、FreeRTOS）

13.3 AOP（面向方面编程）案例补强（2019 真题）

典型题干：日志、安全、事务等横切关注点散落在多个模块中，导致代码重复且难维护，如何重构？

标准答案骨架：

(1) **识别问题**：横切关注点（Cross-Cutting Concerns）跨越多个模块，违反单一职责原则。

(2) **AOP 解决方案**：

- **切面 Aspect**：把日志/事务/安全模块化为切面
- **连接点 Join Point**：方法调用、方法执行
- **切入点 Pointcut**：用表达式匹配需要织入的方法（如 `execution(* com.xx.service.*.*(..))`）
- **通知 Advice**：前置（@Before）、后置（@After）、环绕（@Around）、返回后（@AfterReturning）、异常后（@AfterThrowing）
- **织入 Weaving**：编译时织入（AspectJ） / 运行时织入（Spring AOP 动态代理）

(3) **效果**：

- 业务代码无侵入
- 横切关注点集中维护
- 符合开闭原则与单一职责

(4) 与代理模式关系：Spring AOP 底层 = JDK 动态代理（接口）或 CGLIB（无接口）

13.4 架构演化与治理案例补强

典型题干：系统经多年迭代后出现性能下降、模块耦合严重、技术债累积，如何治理？

标准答案骨架：

(1) 现状评估：

- 架构腐蚀指标：模块耦合度、重复代码率、变更失败率、平均修复时间 MTTR
- 技术债清单：按影响范围 × 修复成本分级

(2) 治理机制：

措施	具体做法	度量指标
ADR 决策留痕	关键技术选型写 Architecture Decision Record	ADR 数量
架构评审	重大变更须经架构委员会评审	评审通过率
质量门禁	CI 中加入静态扫描、单测覆盖率	代码质量分数
技术债台账	列入版本计划逐步偿还	季度清偿率
运行度量	监控变更失败率、MTTR	故障数下降率

(3) 演化路径：

- 识别核心可重构域 → 灰度迁移 → 旧模块下线 → 持续监控

(4) 典型答题句：

"为避免长期演进中的架构腐蚀，我建立了 ADR 机制 + 架构评审 + 质量门禁 + 技术债管理 + 度量闭环的五位一体治理体系，季度技术债清偿率提升 20%，重大故障下降 50%。"

13.5 企业架构（TOGAF）案例补强

典型题干：集团 200+ 业务系统林立、烟囱式建设、数据孤岛严重，要求整体规划。

标准答案骨架：

(1) 现状梳理：

- 系统清单与依赖关系
- 业务流程与组织

- 数据资产分布
- 技术栈与基础设施

(2) 按四层架构设计目标蓝图：

层次	内容	输出物
业务架构	业务能力、流程、组织	业务能力图、流程图
应用架构	应用清单、协作、接口	应用矩阵、接口规范
数据架构	主数据、共享数据、治理	主题域、数据模型
技术架构	部署、网络、中间件、安全	技术栈、部署图

(3) TOGAF ADM 迭代实施： 预备 → A 架构愿景 → B 业务架构 → C 信息系统架构（应用+数据） → D 技术架构 → E 机会与解决方案 → F 迁移规划 → G 实施治理 → H 架构变更管理

(4) 治理机制： 架构评审委员会、技术选型标准、接口规范、主数据治理

(5) 量化效果： 重复建设减少 30%、复用率 +55%、交付周期缩短 40%

13.6 微服务拆分粒度判断（题 2 常考）

题干样式： 给出业务模块清单，要求拆分微服务并说明理由。

拆分原则（DDD 方法）：

1. 战略设计： 识别核心域、支撑域、通用域
2. 限界上下文 Bounded Context： 每个上下文对应 1-2 个微服务
3. 聚合根 Aggregate Root： 维护一致性边界
4. 服务粒度准则：
 - 业务能力单一： 围绕单一业务能力
 - 数据私有： 每服务独立数据库
 - 团队规模： 2-pizza team (< 8 人)
 - 变更频率相近： 高频变更不与稳定模块绑定

常见拆分维度：

- 按业务域： 用户、订单、商品、支付、库存
- 按读写分离： 查询服务 vs 命令服务（CQRS）
- 按变化频率： 稳定核心 vs 活动促销
- 按并发等级： 高并发独立部署

避免的反模式：

- 过细："纳米服务"，运维负担过重
- 过粗：仍是单体只是换名字
- 共享数据库：违反微服务核心原则
- 同步链路过长：A→B→C→D→E（应改异步）

13.7 缓存一致性方案对比（题 2 高频）

方案	写策略	一致性	性能	风险
Cache Aside（旁路）	先更 DB，再删缓存	最终一致	高	极短不一致窗口
Read/Write Through	缓存代理写 DB	强一致	中	缓存层成为瓶颈
Write Behind	异步写 DB	弱	极高	宕机丢数据
双写	同时写 DB + Cache	不可靠	中	不推荐，易不一致

一致性进阶方案：

- 延迟双删：写 DB → 删缓存 → 延迟 N 秒 → 再删一次
- 订阅 binlog：通过 Canal 监听 MySQL binlog 自动更新缓存
- 分布式锁：写操作加锁，串行化更新

13.8 分布式事务方案选型（题 2/题 3 常考）

方案	一致性	性能	复杂度	适用
2PC	强	低（阻塞）	中	跨库事务、传统场景
3PC	强	较低	高	有超时要求
TCC	强	中	高（业务侵入）	资金、订单核心业务
Saga	最终	高	中	长事务（旅行预订）
本地消息表	最终	高	低	通用异步
MQ 事务消息	最终	高	中	RocketMQ 事务消息
Seata AT	强	中	低（无侵入）	中后台一般场景

选型决策树：

是否资金/核心业务? → 是 → TCC / Seata AT

↓ 否

能否最终一致? → 是 → 本地消息表 / MQ 事务消息 / Saga

↓ 否

跨数据库? → 是 → 2PC / Seata

13.9 题 4 类图补全套路（必考）

典型题干：给文字描述与缺失类图，要求补全类、关系、属性、方法。

标准答题步骤：

1. **找名词：**文字中的名词候选为**类或属性**
2. **找动词：**动词候选为**方法或关联**
3. **判断关系：**
 - "X 由 Y 组成" → 组合（同生死） / 聚合（可分离）
 - "X 是一种 Y" → 泛化（继承）
 - "X 实现接口 Y" → 实现
 - "X 使用 Y" → 依赖/关联
4. **标多重性：**1、0..1、1..、
5. **画方法：**动词+宾语，对应业务操作

常见易错点：

- 关联方向写反
- 聚合 vs 组合混淆（**核心是生命周期**）
- 漏掉接口实现关系
- 多重性写在错误一端（注意：多重性写在**对端**）

13.10 题 1 效用树补全多场景示例

案例题 1 经常要求补效用树，下面提供 5 类常见质量属性的标注样例。

电商系统效用树（H/M/L = 重要性, 难度）：

效用 Utility

性能 Performance

- 下单 P99 ≤ 1.5 秒 (H, M)
- 商品页 P99 $\leq 800\text{ms}$ (H, L)
- 大促峰值 50000 TPS (H, H)

可用性 Availability

- 全年可用 99.99% (H, M)
- 单服务故障 30 秒切换 (H, L)
- 异地多活切换 5 分钟 (M, H)

安全性 Security

- 支付链路 TLS 1.3 (H, L)
- 敏感数据 AES-256 加密 (H, L)
- 风控误判率 $< 0.01\%$ (M, H)

可修改性 Modifiability

- 新支付通道接入 ≤ 3 人天 (M, M)
- 促销规则配置化 (H, M)

可测试性 Testability

- 接口自动化覆盖率 $\geq 85\%$ (M, M)

填空套路：

- 题目给场景片段 $\rightarrow$ 标注 (H/M/L, H/M/L)
- 重要性看业务影响、难度看技术挑战
- 高重要性 H** 是必保项，必须先实现

13.11 三种基础架构风格易混点辨析（题 1 常考）

风格	数据耦合	控制耦合	数据流向	典型应用
管道-过滤器	流式	弱（顺序触发）	单向	Unix管道、编译器、ETL
黑板	共享内存	强（控制器协调）	多向（读写共享）	AI、语音识别、IDE
事件驱动	事件消息	弱（发布订阅）	多向（异步）	GUI、消息系统

经典考题判别：

- 题目出现"过滤器""管道""数据流" $\rightarrow$ 管道-过滤器
- 题目出现"知识源""黑板""控制器" $\rightarrow$ 黑板
- 题目出现"事件""订阅""触发""通知" $\rightarrow$ 事件驱动

13.12 案例题常见易错点与失分点

易错点	正确做法
关键词不靠前	答案首句直接给结论，再展开
只写技术名词	写"机制 + 触发条件 + 预期效果"
性能不量化	给具体指标（P99、TPS、RT）
风险无应对	每个风险配 1 条应对措施
不分点	①②③ 或 1./2./3. 编号
答案与题干无关	50% 答案在题干，先标关键词
UML 符号错	类图严格按标准：实/虚线、菱形空心/实心
时间分配失衡	题 1 不超 30 分钟，强制切换

整理日期：2026-05

覆盖范围：近 10 年真题案例分析考点 + 近年新增案例热点补充

使用建议：

- 1. 通读 1 遍标记不熟点
- 2. 每天做 1 套真题，对照本手册查漏
- 3. 题 1 必反复练习（必答题）
- 4. 题 3、题 4 是必拿分点，重点准备
- 5. 考前 7 天按"冲刺清单"过 1 遍
- 6. 配合姊妹篇《[选择题考点速记.md](#)》《[架构设计师考点整理.md](#)》食用更佳

十四、高频"关键词→对应考点"对照表

题干关键词	对应考点
"响应时间"、"吞吐量"、"并发"	性能
"宕机"、"故障"、"恢复"	可用性
"认证"、"授权"、"加密"	安全性
"扩展"、"修改"、"配置"	可修改性
"热点 key"、"穿透"	缓存问题
"拆分服务"、"独立部署"	微服务
"ESB"、"集中通信"	SOA
"数据流"、"过滤器"	管道-过滤器风格
"黑板"、"知识源"	黑板风格
"插件"、"内核"、"扩展"	微内核风格
"无状态"、"资源"、"URI"	REST
"撤销"、"重做"、"队列"	命令模式
"审批流"、"链式"	责任链
"树形"、"部分整体"	组合模式
"代理"、"远程"、"懒加载"	代理模式
"动态"、"增强"、"包装"	装饰器
"通知"、"发布订阅"	观察者
"状态切换"、"行为变化"	状态模式
"周期"、"截止期"、"调度"	实时系统
"传感器"、"网关"、"边缘"	物联网

十五、套装资料一览

文件	用途
架构设计师考点整理.md	综合知识点（含案例与论文）
选择题考点速记.md	上午综合知识 75 题专用
案例分析考点速记.md	下午 I 案例分析专用（本文档）